Documentación Técnica — Kolsor Server

Proyecto: Kolsor
Módulo: C.F.G.S. Desarrollo de Aplicaciones Multiplataforma
Repositorio: https://github.com/rubeneitor21/Kolsor-Server

Índice

1. Descripción general
2. Arquitectura del sistema
3. Estructura del proyecto
4. Stack tecnológico
5. Módulo HTTP y SSR
6. Módulo de autenticación
7. Módulo WebSocket
8. Motor de juego
9. Base de datos
10. Sistema de logs
11. Despliegue
12. Decisiones de diseño

---

1. Descripción general

Kolsor Server es el backend del juego de dados estratégico multijugador Kolsor. Proporciona dos servicios sobre un único proceso Node.js:

- Servidor HTTP para servir la interfaz web de pruebas y las APIs de autenticación.
- Servidor WebSocket para la comunicación en tiempo real del juego: matchmaking, gestión de turnos y lógica de combate.

El cliente principal es una aplicación Unity que se principalmente vía WebSockets, pero tambien hace uso de algunos endpoints HTTP para la autenticacion. La interfaz web existe como herramienta de testing ligero sin necesidad de pasar por el cliente Unity.

---

2. Arquitectura del sistema

```
Cliente Unity / Navegador
        │
        │  HTTP (auth, páginas)
        │  WS (juego en tiempo real)
        ▼
┌──────────────────────────────┐
│         Node.js Server       │
│                              │
│  ┌──────────┐  ┌──────────┐  │
│  │  HTTP    │  │    WSS   │  │
│  │ Server   │  │  Server  │  │
│  └──────────┘  └──────────┘  │
│       │             │        │
│  ┌────▼─────────────▼──────┐ │
│  │       SSR / Router      │ │
│  │   Api/ · Pages/ · pub/  │ │
│  └─────────────────────────┘ │
│               │              │
│  ┌────────────▼────────────┐ │
│  │   Game Server           │ │
│  │  commands · Room · RNG  │ │
│  └─────────────────────────┘ │
│               │              │
│  ┌────────────▼────────────┐ │
│  │      Database           │ │
│  │    (MongoDB driver)     │ │
│  └─────────────────────────┘ │
└──────────────────────────────┘
        │
        ▼
    MongoDB
```

El servidor HTTP y el WebSocket comparten el mismo proceso y escuchan en el mismo puerto. El servidor HTTP se crea primero y se pasa como parámetro al `WebSocketServer`, que lo usa como transporte.

---

3. Estructura del proyecto

```
Kolsor-Server/
├── src/
│   ├── main.ts                     # Punto de entrada, setup HTTP + WS
│   └── utils/
│       ├── SSR.tsx                 # Router HTTP y renderizado React
server-side
```

```
│       ├── database.ts            # Capa de acceso a MongoDB
│       ├── logger.ts              # Sistema de logs propio
│       ├── httpMessages.ts        # Utilidades de parsing de requests
│       └── gameServer/
│           ├── commands.ts        # Comandos WS y lógica de Room
│           └── dice.ts            # Motor RNG determinista
├── Api/
│   ├── login.ts                   # POST /api/login
│   └── register.ts                # POST /api/register
├── Pages/
│   ├── index.tsx                  # Página principal (protegida)
│   ├── 404.tsx                    # Página de error
│   └── auth/
│       ├── login.tsx              # Página de login
│       └── register.tsx           # Página de registro
├── components/
│   └── Layout.tsx                 # Layout React compartido
├── public/
│   ├── game/main.js               # Cliente web de testing
│   ├── auth/login.js              # Lógica del formulario de login
│   └── auth/register.js           # Lógica del formulario de registro
├── types/                         # Declaraciones TypeScript globales
├── scripts/
│   └── build-routes.js            # Generador de routeMap estático
├── docker-compose.yml
├── build.mjs
└── tsconfig.json
```

---

4. Stack tecnológico

Categoría	Tecnología	Justificación
Runtime	Node.js 20	LTS estable, soporte nativo de WebSockets
Lenguaje	TypeScript 5	Tipado estático, mejor mantenibilidad
WebSocket	`ws` 8	Librería directa sin abstracciones, menos problemas de integración que frameworks
HTTP	`node:http` nativo	El HTTP es infraestructura mínima; añadir Express no aporta funcionalidad necesaria
Renderizado	React 19 + `react-dom/server`	SSR para las páginas web sin dependencia de frameworks como

Categoría	Tecnología	Justificación
		Astro o Next que complican la integración WS
Base de datos	MongoDB 7	Esquema flexible para el modelo de usuario en evolución
Autenticación	JWT (`jsonwebtoken`) + bcrypt	Estándar para APIs stateless, compatible con cliente Unity
RNG	`seedrandom`	PRNG determinista verificable por ambos jugadores
Logs	Implementación propia	Control total del formato y gestión de backpressure
Despliegue	Docker + docker-compose	Portabilidad y gestión de servicios con MongoDB incluido

---

5. Módulo HTTP y SSR

Router (`src/utils/SSR.tsx`)

El router implementa un sistema de file-based routing similar a Next.js. Las peticiones se resuelven en tres categorías:

1. **Archivos estáticos** (`public/`): se sirven directamente con detección de MIME type via `mime-types`.
2. **API routes** (`/api/*`): se mapean a módulos en `Api/` y se despachan por método HTTP (`GET`, `POST` ...).
3. **Páginas** (`Pages/`): se renderizan con React SSR mediante `renderToString`.

Optimización en producción: El script `scripts/build-routes.js` se ejecuta antes del build y genera un fichero `src/routeMap.ts` con todos los imports estáticos de las páginas y APIs. En producción, el router usa este mapa y evita `import()` dinámicos y llamadas a `existsSync` en el camino caliente de cada petición, lo que facilita el minimizado de la aplicación, así como su fácil distribución.

```
Build time:
  build-routes.js → src/routeMap.ts (imports estáticos)

Runtime producción:
  petición → routeMap lookup (O(1)) → handler
```

```
Runtime desarrollo:
  petición → existsSync + dynamic import (fallback)
```

Protección de rutas

La función `isAuthenticated(req)` lee la cookie `token` de los headers HTTP y verifica su presencia. La ruta `/` devuelve un 302 a `/auth/login` si no hay cookie. La verificación criptográfica del JWT ocurre en `main.ts` durante el handshake WS, no en el SSR.

Propagación de headers

Los endpoints pueden devolver headers opcionales en su respuesta (`EndpointResponse.headers`). El SSR los propaga a `FrontResponse.headers` y `main.ts` los aplica antes de `res.end()`. Esto permite que el login setee la cookie HttpOnly desde el servidor. Aunque de momento la interfaz web solo es para testing, se espera ampliar a obtener diferentes estadísticas e implementar de un cliente ligero.

---

6. Módulo de autenticación

Flujo de registro (`POST /api/register`)

```
Cliente → POST /api/register { username, password }
  → validateFields (campos requeridos, no vacíos)
  → bcrypt.hash(password, 10)
  → MongoDB insertOne
  → { id } | { error }
```

Flujo de login (`POST /api/login`)

```
Cliente → POST /api/login { username, password }
  → validateFields
  → MongoDB findOne por username
  → bcrypt.compare(password, hash ?? DUMMY_HASH)  ← timing-safe
  → jwt.sign({ id, username }, JWT_SECRET, { expiresIn: '24h' })
  → { token, user } + Set-Cookie: token=...; HttpOnly; SameSite=Strict
```

Protección de timing: Cuando el usuario no existe, se compara la contraseña contra `DUMMY_HASH` (un hash bcrypt real pre-generado) en lugar de devolver inmediatamente. Esto garantiza que el tiempo de respuesta sea siempre el coste de un `bcrypt.compare` (~100ms), independientemente de si el usuario existe o no, eliminando el side-channel que permitiría enumerar usernames por tiempo de respuesta.

Cookie HttpOnly

El token se devuelve en el body de la respuesta (para el cliente Unity) y simultáneamente se setea como cookie HttpOnly (para el cliente web). La cookie incluye `SameSite=Strict` y `Secure` en producción.

```
Set-Cookie: token=<JWT>; HttpOnly; SameSite=Strict; Secure; Path=/; Max-Age=86400
```

El cliente web no necesita gestionar el token manualmente, el navegador lo adjunta automáticamente en las peticiones HTTP y en el handshake WS.

Autenticación WebSocket

Al establecer la conexión WS, `main.ts` extrae la cookie `token` de los headers del handshake HTTP y verifica el JWT. Si es válido, el cliente se registra directamente con su ID real. Si no, se le asigna un UUID temporal hasta que mande el comando `auth`.

---

7. Módulo WebSocket

Gestión de clientes

```
const clients: Map<string, WebSocket> = new Map()
```

Cada conexión se identifica inicialmente con un UUID temporal o el ID del JWT si la cookie estaba presente. El mapa relaciona ID → socket activo.

UUID dual: Al conectarse, se guarda tanto el UUID temporal (`firstUuid`) como el UUID real post-autenticación (`uuid`). El handler `onclose` limpia ambos para evitar entradas huérfanas en el mapa.

Sistema de keepalive (ping/pong)

Cada conexión tiene un timer de 6 segundos. El cliente debe mandar `ping` periódicamente para reiniciarlo. Si el timer expira, `closeTimeout` envía un aviso y cierra la conexión. El timer se actualiza correctamente al reasignar el UUID tras autenticación.

```
connection → timer(6s)
ping       → clearTimeout + reset timer(6s) + pong
```

```
auth       → clearTimeout(UUID_temp) + nuevo timer(6s, UUID_real)
timeout    → aviso + ws.close() + limpieza de pings y clients
```

Protocolo de comandos

Los mensajes WS siguen la estructura:

```
{ "type": "nombre-comando", "body": { ... } }
```

El campo `from` lo inyecta el servidor (no el cliente) para garantizar que el origen sea siempre el ID verificado. `processCommand` despacha al handler correspondiente del registro `responseCommands`.

Comando	Dirección	Descripción
`connection`	servidor	Registro inicial y setup del timer
`ping`	cliente → servidor	Keepalive
`pong`	servidor → cliente	Respuesta con timestamp
`auth`	cliente → servidor	Autenticación por JWT (clientes sin cookie)
`matchmaking-search`	cliente → servidor	Buscar/crear sala
`matchmaking-join`	servidor → cliente	Confirmación de sala
`game-start`	servidor → cliente	Inicio de partida con jugadores y orden
`game-rolls`	servidor → cliente	Dados disponibles para el jugador activo
`select-rolls`	cliente → servidor	Dados seleccionados por el jugador
`god-favor`	servidor → cliente	Fase de elección de favor divino
`select-favor`	cliente → servidor	Elección de favor del jugador
`resolution-attack-first`	servidor → cliente	Resultado del ataque del primer jugador
`resolution-attack-second`	servidor → cliente	Resultado del ataque del segundo jugador
`game-over`	servidor → cliente	Fin de partida con ganador

Comando	Dirección	Descripción
`closing`	servidor → cliente	Aviso de cierre inminente

---

8. Motor de juego

Matchmaking

El sistema busca salas públicas con espacio usando un `for...of`. Si no hay sala disponible, crea una nueva. En un futuro esto debería cambiarse por una búsqueda en la base de datos o en una solicitud a un servidor maestro para permitir escalado horizontal, permitiendo el indexado de partidas por llenar reduciendo tiempos de búsqueda

```
const clientsRoom: Map<string, Room> = new Map()  // UUID → Room
const rooms: Map<string, Room> = new Map()         // roomId → Room
```

`clientsRoom` almacena la referencia directa al objeto `Room` para que los handlers de juego puedan llegar a la sala en O(1) desde el UUID del jugador.

Clase Room

Encapsula todo el estado de una partida y su lógica. Los métodos públicos son los puntos de entrada desde los handlers WS:

- `addUser(id, ws)` — añade jugador, inicia partida al llegar el segundo
- `updateRolls(user, rolls)` — procesa la selección de un dado
- `updateGodFavor(user, favor)` — procesa la elección de favor divino

El estado de la sala sigue esta máquina de estados:

```
not-started → select-rolls ⇄ god-favor → select-rolls (nuevo turno)
                                        → game-over
```

La maquina de estados tiene una implementación pobre, sobretodo en cuanto a transiciones entre los mismos. Se espera un refactor total de la clase si el tiempo lo permite, ya que hay otros aspectos mas importantes por el momento

Mecánica de turnos

Cada turno se divide en tres fases:

1. Selección de dados (select-rolls): Hay hasta 3 rondas de selección. En cada ronda el jugador activo escoge un dado. Los turnos alternan entre jugadores empezando por `playerStart`. Se lanzan 6 dados menos los ya seleccionados por el jugador activo, al finalizar la 3 ronda, se seleccionan todos los dados restantes de forma automatica.

2. Favor divino (god-favor): Cuando se han seleccionado 12 dados en total (6 por jugador) o se han completado 3 rondas. Cada jugador elige un favor. La fase avanza cuando ambos han elegido.

3. Resolución (resolution): Se procesan todos los dados seleccionados y se aplican los favores:

```
Para cada jugador:
  aDistancia = flechas
  dDistancia = cascos
  aMelee     = hachas
  dMelee     = escudos
  steal      = manos

Robo de energía (playerStart → playerSecond):
  energía_robada = min(energía_rival, steal_propio)

Daño recibido por playerSecond:
  daño = max(0, aDistancia - dDistancia_rival)
       + max(0, aMelee - dMelee_rival)

[ igual para playerSecond → playerStart ]
```

Tras la resolución, si ningún jugador ha llegado a 0 de vida, se intercambian `playerStart` y `playerSecond` y comienza un nuevo turno.

La implementación actual de resolución tiene una gran parte de código duplicado, por lo que se buscara una forma mas limpia de implementarlo, ademas, el objeto (diccionario) en uso ahora mismo fuerza la comprobación manual de la cara, por lo que sera sustituido por el string asignado al propio enumerador

RNG determinista (`src/utils/gameServer/dice.ts`)

El motor de aleatoriedad usa `seedrandom` con una semilla construida a partir de los IDs de los dos jugadores concatenados con el ID de la sala. Esto garantiza que la secuencia de dados es reproducible y verificable, pero tampoco permite que se abuse por un tercero.

```
const seed = [...this.users.keys()].reduce((a, b) => a + b) + this.id
this.rng = new DiceRNG(seed, this.id)
```

Los primeros N valores (N entre 0 y 19) se descartan en el constructor para mitigar la baja entropía inicial de algunos PRNGs, el valor es generado de forma aleatoria para dificultar que el usuario pueda predecir todas las tiradas de antemano.

Cada tirada se genera y codifica en 4 bits:

```
bits[3]   = energía (probabilidad 0.4)
bits[2:0] = cara (0-4 → AXE, ARROW, HELMET, SHIELD, HAND)
```

Las 5 caras del dado:

Cara	Tipo	Efecto en resolución
AXE (hacha)	Ataque melee	Daño reducido por escudos rivales
ARROW (flecha)	Ataque distancia	Daño reducido por cascos rivales
HELMET (casco)	Defensa distancia	Reduce daño de flechas recibidas
SHIELD (escudo)	Defensa melee	Reduce daño de hachas recibidas
HAND (mano)	Robo	Roba energía al rival

---

9. Base de datos

Conexión

`Database` es un singleton que gestiona una única instancia de `MongoClient`. El método `init()` establece la conexión y realiza un ping de verificación, devolviendo `"connected"` o `"error"`. El servidor no empieza a escuchar hasta que `init()` resuelve con éxito.

Colecciones

`users`:

```
{
  _id: ObjectId,
  username: string,       // índice único
```

```
  password: string,       // hash bcrypt
  level: number,          // nivel del jugador
  exp: number,            // Experiencia actual
  level-exp: number,      // Cantidad de experiencia necesaria para subir
de nivel
  stats: {
    wins: number,
    losses: number,
    disconnections: number
  },
  createdAt: Date
}
```

Seguridad

- Las contraseñas se almacenan siempre como hash bcrypt con coste 10.
- Los IDs de usuario se validan con `ObjectId.isValid()` antes de construir el `ObjectId` para evitar `BSONError`.
- Las consultas de login devuelven siempre "Credenciales inválidas" sin distinguir usuario inexistente de contraseña incorrecta.

---

10. Sistema de logs

El `Logger` es un singleton con dos salidas simultáneas: consola (solo en desarrollo, con colores via `chalk`) y fichero (`latest.log`, sobreescrito en cada arranque).

Gestiona backpressure del stream de escritura: si el buffer interno está lleno (`write()` devuelve `false`), espera el evento `drain` antes de continuar y registra el tiempo de pausa en consola.

En producción (`NODE_ENV !== "development"`) los logs de consola están desactivados para no interferir con el stdout del contenedor Docker.

Niveles disponibles: `INFO`, `WARNING`, `ERROR`.

---

11. Despliegue

Variables de entorno

```
PORT=3000
NODE_ENV=production
JWT_SECRET=<secreto-aleatorio-seguro>   # Obligatorio. El servidor no
```

```
arranca sin él.
DB_CONNECTION=mongodb://user:pass@host/db?authSource=admin
```

Docker Compose

El `docker-compose.yml` levanta dos servicios:

- **kolsor:** Node.js 20 Alpine, ejecuta el build compilado (`node dist/main.js`).
- **mongo:** MongoDB con volumen persistente para los datos.

```
# Build del proyecto
npm run build

# Arrancar servicios
docker-compose up -d
```

El servicio `kolsor` espera que el `dist/` esté disponible en el volumen montado. El build debe ejecutarse antes de levantar los contenedores. El comando docker-compose es compatible tanto con docker-compose como con docker compose, ademas ambos deben usarse con las variables de entorno `JWT_SECRET` y `MONGO_PASSWORD` setteadas. En entornos Linux puede levantarse con el siguiente comando

```
JWT_SECRET=token_seguro MONGO_PASSWORD=contraseña_segura docker compose up
-d
```

Scripts

Script	Descripción
`npm run dev`	Modo desarrollo con `nodemon`, recarga en cambios de `.ts/.tsx/.d.ts`
`npm run build`	Genera `routeMap.ts` + compila con esbuild + limpia artefactos temporales
`npm start`	Ejecuta el build compilado

---

12. Decisiones de diseño

HTTP nativo en lugar de Express: La capa HTTP gestiona únicamente autenticación y servir archivos estáticos. El núcleo del servidor es WebSocket. Añadir Express introduciría una dependencia con su propio

modelo de middleware que no aporta funcionalidad necesaria para este caso de uso.

Sin framework frontend (Astro, Next): Los frameworks modernos de SSR tienen abstracciones pensadas para request/response que complican la integración de WebSockets. Al implementar el SSR directamente con `react-dom/server`, el servidor HTTP y el WS comparten proceso y configuración sin fricciones. Ademas añaden tanto dependencias innecesarias como un tamaño de bundle mucho mayor al realmente utilizado

File-based routing propio: El sistema de routing se inspira en Next.js pero se genera en build time en lugar de resolverse dinámicamente. En producción, cada petición es un lookup en un objeto con los módulos ya importados, sin accesos al sistema de ficheros.

RNG determinista con semilla compartida: El servidor es la única fuente de verdad para los dados. La semilla se construye a partir de información que ambos jugadores conocen (sus IDs y el ID de la sala), lo que permite auditar cualquier partida reproduciendo la secuencia completa. El cliente nunca genera números aleatorios de juego.

Estado de partida centralizado en Room: Todo el estado mutable de una partida (vida, energía, dados seleccionados, ronda, fase) vive exclusivamente en el objeto `Room` en servidor. El cliente es un presentador que recibe el estado y envía acciones, nunca toma decisiones sobre el resultado de esas acciones.